

Testing, Debugging, and Deployment

Given a scenario, formulate the deployment process, supporting tools, and mechanisms for source-driven development.

Table of Contents

-  [Deployment Process](#)
-  [Change Set Development Model](#)
-  [Org Development Model](#)
-  [Package Development Model](#)
-  [Salesforce DX Concepts](#)
-  [Visual Studio Code](#)
-  [Code Builder](#)
-  [Local Dev](#)



Table of Contents

-  [Scratch Orgs](#)
-  [Salesforce CLI](#)
-  [Partners, Packaging, and AppExchange](#)
-  [DevOps Center](#)
-  [Scenarios and Solutions](#)



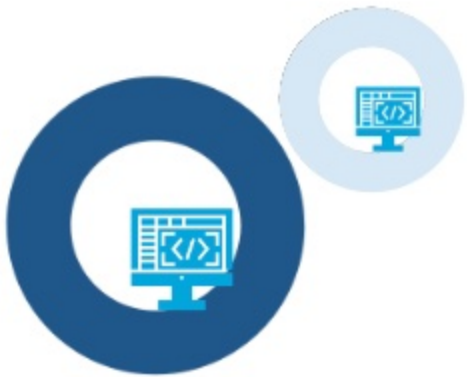
After studying this topic, you should be able to:

-  Identify different development models and deployment tools used for migrating metadata.
-  Describe concepts related to source-driven development and tools that can be used to support the process.
-  Identify the appropriate deployment tool for different deployment scenarios.



Introduction

Salesforce provides **tools** that allow developers to **deploy changes** from one environment to another. Depending on the **scenario**, or **phase** in the deployment process, a particular tool is needed to perform a certain task. In this topic, we describe a **deployment process** and its **supporting tools**, as well as the **deployment options** to use in a given scenario.



Deployment Tool Options

The following tools are capable of **deploying metadata** to org environments.

Salesforce Extensions for Visual Studio Code

Visual Studio Code extensions that empower the editor for Salesforce development and deployment of metadata to any org.

Code Builder

A web-based integrated development environment optimized for development and does not require any installation.

Salesforce CLI

A command-line interface that uses the metadata API for development, deployment and other tasks in the development lifecycle.



Change Sets

Deployment tool used for migrating metadata between related orgs such as a sandbox org and a production org.

Developer Console

A native integrated development environment for developing and deploying metadata to its associated org.

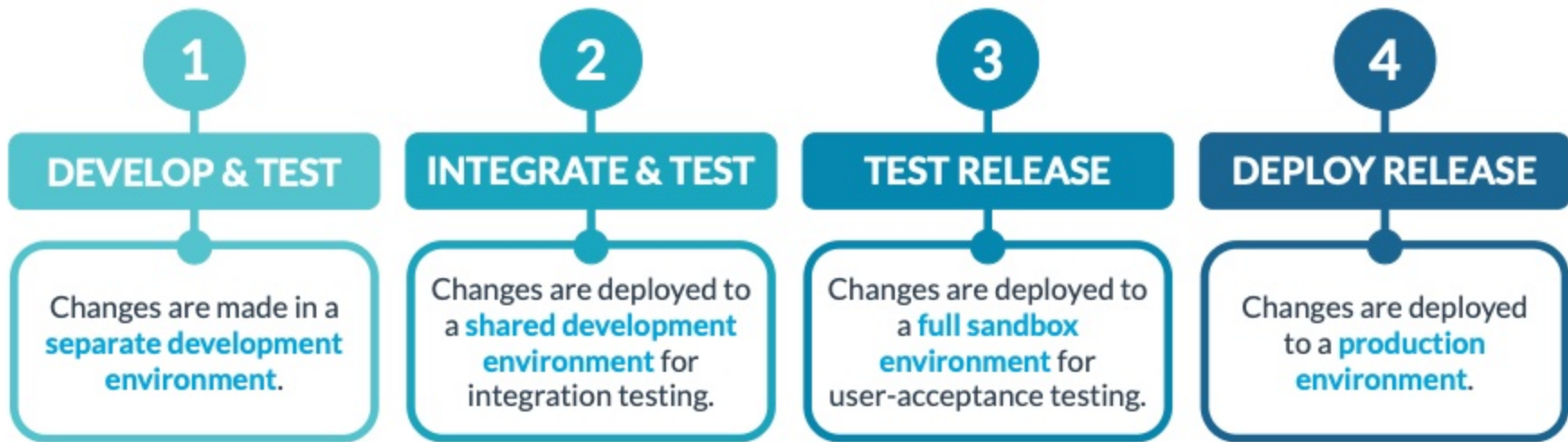
Deployment Process

Table of Contents



Deployment Process

A typical deployment process **consists** of the following **phases**:



Deployment Process

One of the following development models can be utilized for the **development** and **deployment** of metadata in Salesforce:



Change Set Development
Model



Org Development
Model



Package Development
Model

Change Set Development Model

[Table of Contents](#)



Change Set Development Model

The change set development model involves the **development** and **deployment** of Salesforce customizations using **change sets**. This model consists of the following steps:

1

DEVELOPMENT SANDBOXES

A **separate** Developer sandbox is created for **each developer** to ensure that the changes remain **isolated** until the team is ready to **integrate** them.

2

DEVELOP AND TEST

The developers use their Developer sandboxes to **create** and **test customizations** based on business requirements. The sandboxes **do not** contain production data.

3

INTEGRATION TESTING

Change sets are used to **migrate customizations** from the **Developer** sandboxes to a shared **Developer Pro** sandbox for integration.

4

USER-ACCEPTANCE TESTING

Change sets are used to migrate customizations to a **full sandbox** which is used for user acceptance testing. The full sandbox contains all the **production data**.

5

PRODUCTION RELEASE

The customizations are migrated to the **production environment** using change sets. The full sandbox can be used for end-user training.

Change Set Development Model Considerations

Some of the main **considerations** related to the change set development model are as follows:



TRACKING CHANGES

A **change-tracking method** needs to be **established** for tracking changes, especially for changes that require manual migration.



USE TRACKING TOOLS

A **custom** or **third-party tool** can be utilized by developers to log each change made by them.



MANUAL CONFIGURATION

Configurations involving metadata that are **not supported** by the Metadata API require recreating the customization **manually** in the destination org.



METADATA COVERAGE REPORT

The Metadata Coverage report shows which **metadata types** are supported in the Metadata API and other metadata channels.

Using Change Sets

Change sets contain a collection of **metadata** that can be **deployed** from one org to another using the Salesforce **user interface**.

◆ DECLARATIVE TOOL

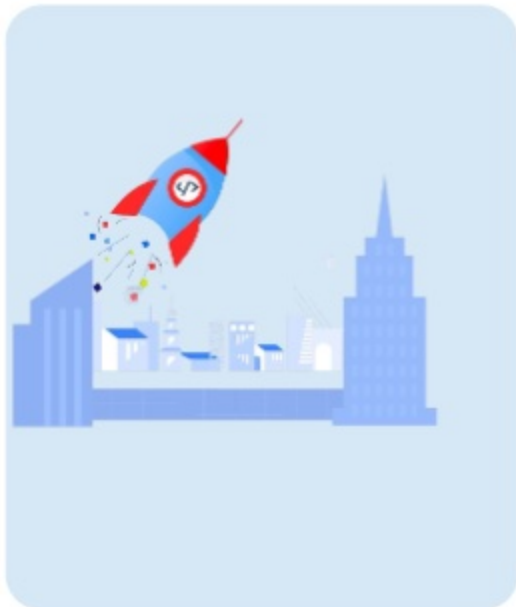
Change sets utilize an **easy-to-use user interface** in Salesforce for small deployments.

◆ NO LOCAL FILES

They allow change management **without using a local file system**.

◆ RELATED ORGS

Metadata can be deployed **between related sandbox/production orgs**.



Using Change Sets

❖ AUDIT CHANGES

Previously deployed changes **can be audited**.

❖ MULTIPLE TARGETS

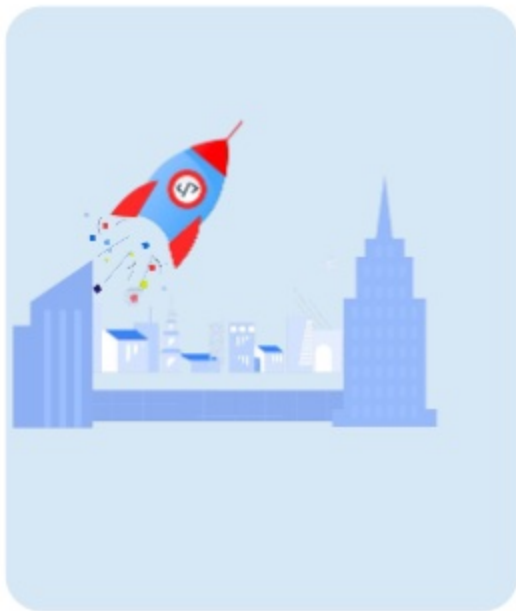
The same component can be **deployed to multiple orgs**.

❖ DELETE / RENAME

It is **not possible** to delete or rename **components** using change sets.

❖ VERSION CONTROL

Change sets **do not support** version control systems.



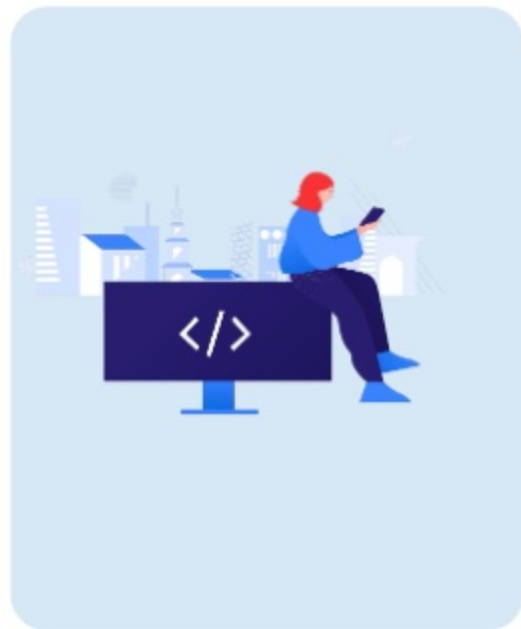
Org Development Model

Table of Contents



Org Development Model

The org development model **involves** the following development and deployment **aspects**:



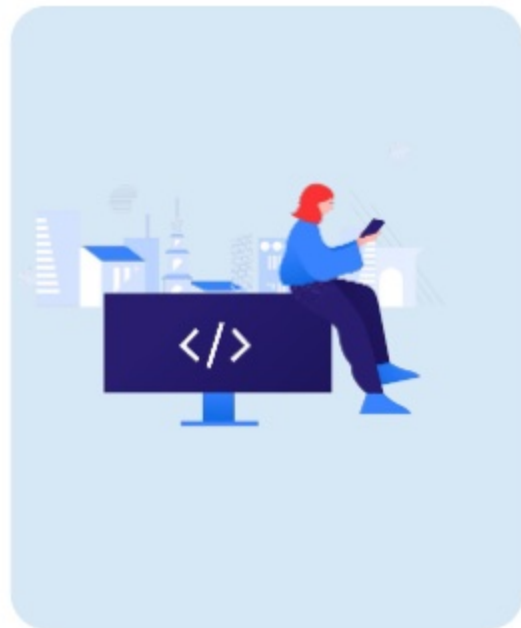
❖ SALESFORCE DX

A Salesforce DX project is created to contain **source code** and **metadata files**.

❖ SOURCE CONTROL

A source control system is used to **manage**, **merge**, and **track** changes in a code repository.

Org Development Model



◆ MULTIPLE ORGS

Multiple environments are set up to handle the **different phases** of the **application development life cycle**.

◆ SALESFORCE CLI

Salesforce CLI is used in each phase of the application development lifecycle such as **retrieving source files** from the org or **deploying changes** to the target org.

Change Management Mechanisms

In this development model, the following can be integrated in a **change management system** to facilitate in **tracking** and **managing** changes in the org.



CHANGE LIST

A list, table, or spreadsheet can be used to **track the changes** made in the org to determine which **customizations to externalize**.



DEPLOYMENT RUN LIST

A list can be used to track and manage the changes that need to be **deployed manually** such as profile and permission sets.



PROJECT MANAGEMENT TOOLS

Tools such as Agile Accelerator and Jira facilitate in implementing **agile development** processes to manage development lifecycles **efficiently**.

Org Development Model

The **development** and **deployment** process in this model consists of the following phases:

1

DEVELOP AND TEST

Changes are made in a **separate development environment** such as a Developer sandbox with no production data.

2

BUILD RELEASE

Changes are deployed to a shared development environment for **integration testing**.

3

TEST RELEASE

Changes are deployed to a sandbox environment for **user acceptance testing**.

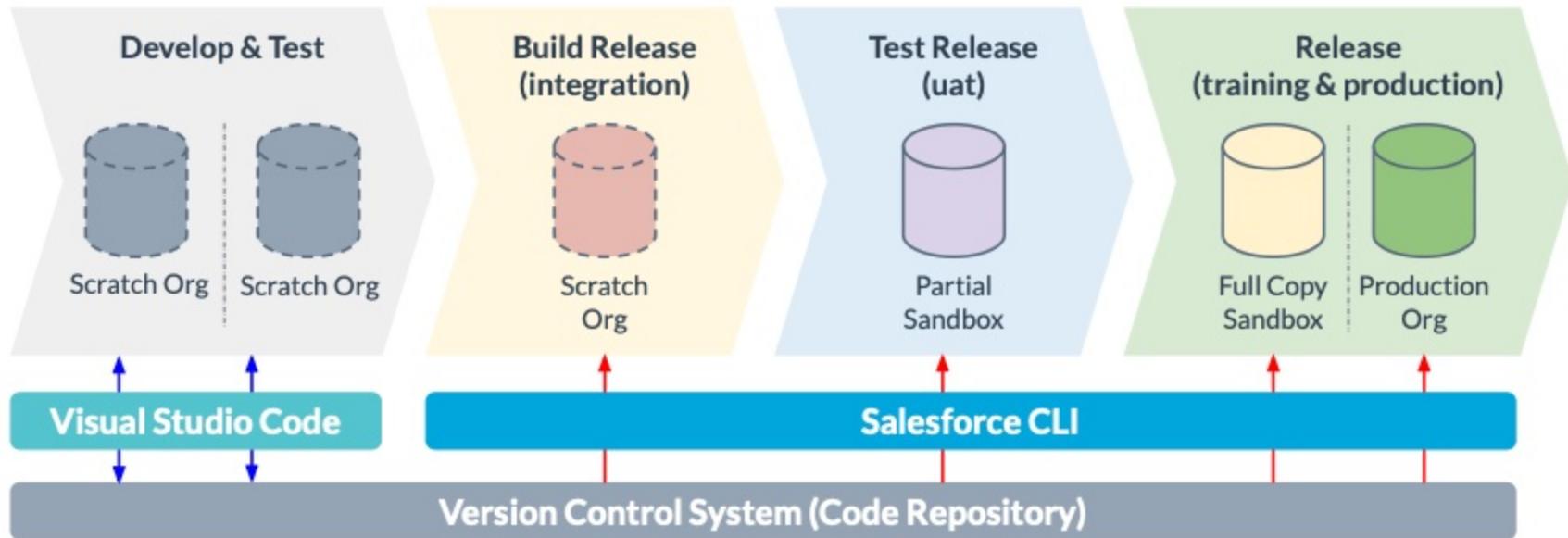
4

DEPLOY RELEASE

Changes are deployed to a **production environment**.

Sample Flow in Org Development Model

In this model, developers use **scratch orgs** and **Visual Studio Code**, for example. Releases are deployed into the integration, uat, and training sandboxes and production org using **Salesforce CLI**.



Package Development Model

[Table of Contents](#)



Package Development Model

The package development model involves creating a **package** of **customizations** that is deployed to the production environment.

❖ PACKAGE

A package is a **release artifact** that contains a group of **related code** and **customizations**.

❖ PACKAGE CONTENT

Source and **metadata** are organized into a package.

❖ VERSION CONTROL

Versioning is used to help with change management. A **version control system** plays an integral role in the package development life cycle.



Package Development Model

❖ CODE REPOSITORY

A source control repository is used to store the **source** for the packages and maintain the **source of truth**.

❖ SCRATCH ORGS

Scratch (development) orgs can be built from the **source** in the **repository** to work specifically on the package.

❖ SALESFORCE DX

Salesforce DX can be used to **test**, **build**, and **release** packages. A Salesforce DX project is used to **organize** the source into a package directory.



Package Development Model

❖ SYNCHRONIZATION

Synchronization of the version control system is improved through **change tracking** of Setup features.

❖ SALESFORCE CLI

Automated testing, **continuous integration**, and **continuous delivery** are typically utilized in the package development life cycle. Salesforce CLI can be used throughout the life cycle.

❖ UNLOCKED PACKAGES

Enterprise edition customers can utilize unlocked packages to **develop** internal business apps.



Package Development Model

❖ VERSION REPORT

The Salesforce CLI `force:package:version:report` command can be used to retrieve details of a specific version of a **managed 2GP** or **unlocked package**.

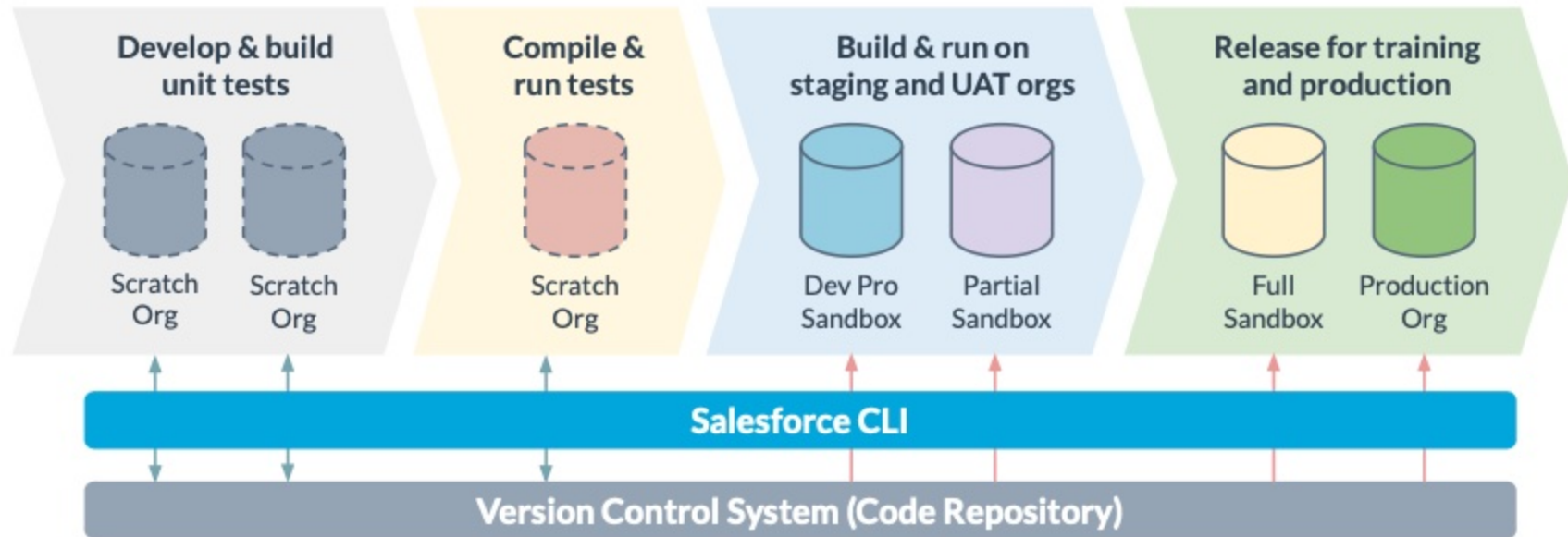
❖ CODE COVERAGE

When adding the `--verbose` flag to the `force:package:version:report` command, the **code coverage percentage** of each Apex class contained in the managed 2GP and unlocked package is included in the output.



Sample Flow in a Package Development Model

Activities related to source development are performed in **scratch orgs**, whereas release and deployment testing activities are performed in **sandbox orgs**.



Salesforce DX Concepts

Table of Contents



What is Salesforce DX

Salesforce DX (*Developer Experience*) introduces a new way of **developing and managing applications** on the Lightning Platform throughout the **entire life cycle of an application**.

Key Characteristics of Salesforce DX:

◆ SOURCE-DRIVEN

It promotes source-driven **development**, **team collaboration**, and **custom app development agility** in Salesforce.

◆ DEVELOPER TOOLS

Tools that developers are **already familiar with** such as ANT or third-party IDEs can be used in Salesforce DX

◆ SOURCE OF TRUTH

Source code and metadata is stored outside the org and managed using a **version control system** such as GIT



What is Salesforce DX

❖ SALESFORCE CLI

Salesforce **command-line interface (CLI)** is used for development, continuous integration, and delivery

❖ SCRATCH ORGS

Scratch orgs are used for **development** and help facilitate **automated testing** and **continuous integration**

❖ CODE EDITOR

Enable **any IDE or text editor** to work with Salesforce CLI and version control system

❖ EDITOR PLUGIN

Salesforce Extensions for VS Code is used for development and working with scratch orgs, Apex, Lightning components, and Visualforce



Salesforce DX and Packaging

Managed and unmanaged packages belong to **first-generation packaging**. Salesforce DX introduced **second-generation packaging** which offers a new approach to packaging and deploying applications.



1ST-GENERATION PACKAGING (1GP)

Managed packages and unmanaged packages are **created and deployed from an org**



2ND-GENERATION PACKAGING (2GP)

Managed 2G packages and unlocked packages are **created and deployed from a local machine or repository**

Salesforce DX and Packaging



NOTE



Managed 2GPs are used for distributing apps on AppExchange. **Unlocked packages** provide more features compared to unmanaged packages such as version upgrades/downgrades, destructive changes, and more.

Environment Setup for Salesforce DX

To start working with Salesforce DX, the following **configurations are required**:



ENABLE DEV HUB

Enable the Developer Hub in the org (requires Production or Developer Edition org) in order to **track the number and status of scratch orgs**.



ADD USERS IN DEV HUB

Create additional users in the Dev Hub org to allow members in a team **to generate scratch orgs** if required



ENABLE 2GP PACKAGING

Enable *Second-Generation Packaging* in the org to develop **2GP managed packages** if required



INSTALL SALESFORCE CLI

Salesforce CLI must be installed in the local development machine in order to **create environments, synchronize code, and execute tests**

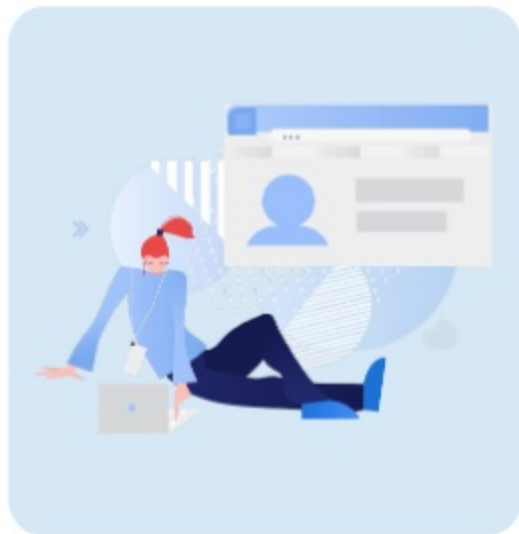
Salesforce DX Project

Salesforce DX implements a new **project structure** to contain org metadata, templates, sample data, and tests. A **Salesforce DX project** is a local directory structure of your metadata in **source format**.

❖ PROJECT FILES

Project files should be stored in a **version control system** (VCS) and retrieved during development and may also contain:

- ❖ Configuration files for **creating scratch orgs**
- ❖ Data to be loaded into orgs for **development or testing**
- ❖ Tests that are created to **validate a package**



Salesforce DX Project

To **set up** a Salesforce DX Project, either of the following can be performed:

❖ USE THE SAMPLE REPOSITORY ON GITHUB

Clone the sample GitHub repo provided by Salesforce as a starting point

❖ CREATE A SALESFORCE DX PROJECT FROM EXISTING SOURCE

Start with an existing Salesforce app to **create a Salesforce DX project and retrieve existing source code**

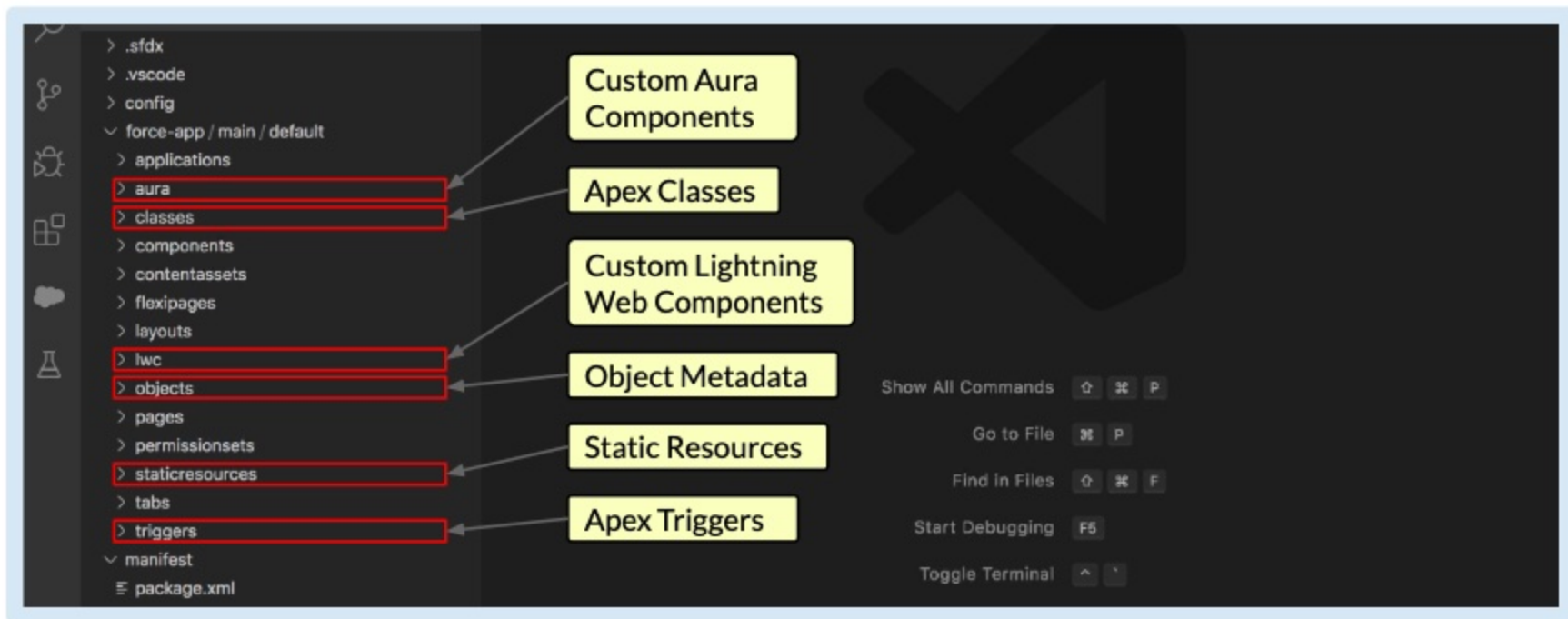
❖ SET UP A FRESH SALESFORCE DX PROJECT

Create an app on the Lightning Platform from scratch using a Salesforce DX project



Example Salesforce DX Project

The following shows an example **project directory structure** used in Salesforce DX.



Visual Studio Code

Table of Contents



Visual Studio Code (VS Code)

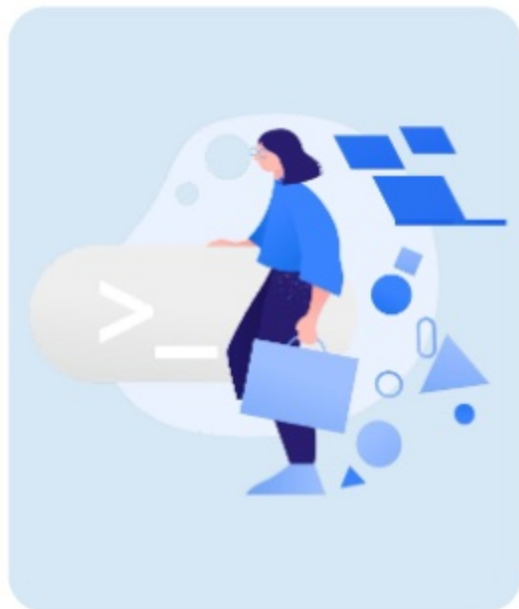
Visual Studio Code is a **source-code editor** that can be used for **development**, **testing**, and **deployment** of Salesforce applications.

❖ REQUIREMENTS

Visual Studio Code requires the installation of **Salesforce CLI** and the **Salesforce Extension Pack** on the local machine.

❖ SALESFORCE DX

Visual Studio Code can be used to **build** and test **applications** in Salesforce using Salesforce DX projects.



Salesforce Extensions for Visual Studio Code

The Salesforce Extension pack is built on top of **Salesforce CLI** and **Visual Studio Code**, transforming the editor into an **integrated development environment** for custom development. Its **extensions** are:



SALESFORCE CLI

Provides **integration** with Salesforce CLI



APEX REPLAY DEBUGGER

Replay **Apex execution** from Apex Debug Log



APEX LANGUAGE

Provides **code-editing features** for the Apex programming language



LIGHTNING WEB COMPONENTS

Provides **code-editing features** for Lightning Web Components



AURA COMPONENTS

Provides **code-editing features** for Aura Components



VISUALFORCE

Provides **syntax highlighting** for the Visualforce framework

Visual Studio Code (VS Code)

There are certain **considerations** for using Visual Studio Code for **deployment**:

LOGIN



Logging in is required to establish a **connection** between **Visual Studio Code** and a **target org**. The command 'SFDX: Authorize an Org' can be used, or it can be selected from the command palette.

ORG ALIAS



An **alias** is created for **each org** with which Visual Studio Code has established a **connection**.

PULL FROM REPO



Code can be pulled from a **git repository** to update the **code base** on the local machine.

Visual Studio Code (VS Code)



PULL FROM ORG

Code can also be pulled from an **org** by using the **command**, 'SFDX: Retrieve Source from Org'.



DEPLOY TO ORG

Code can be deployed using the **command**, 'force:source:deploy' (SFDX: Deploy Source to Org) or 'force:mdapi:deploy'.

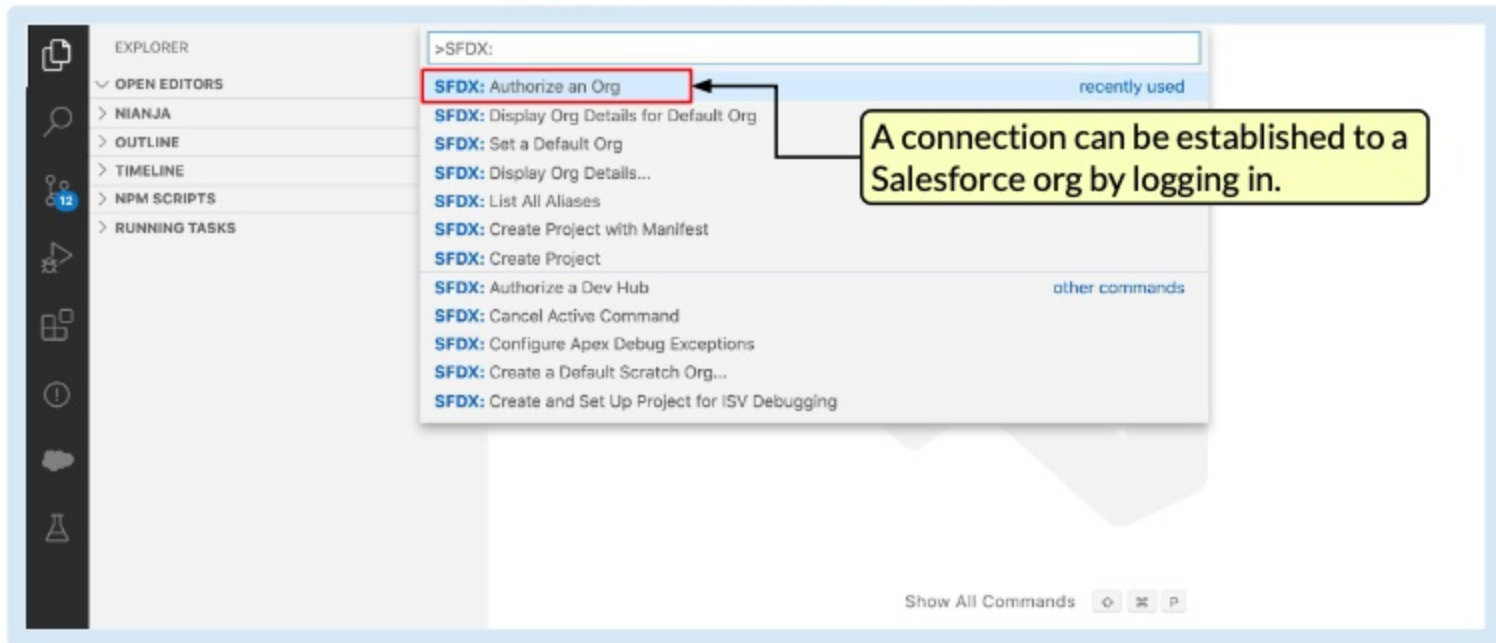


TRACK CHANGES

Changes between a **local project** and **sandbox org** can be determined by using the 'force:source:status' **command**. This requires the 'Enable Source Tracking in this Sandbox' setting to be enabled in the sandbox org. Note that source tracking is only supported in Developer and Developer Pro sandbox.

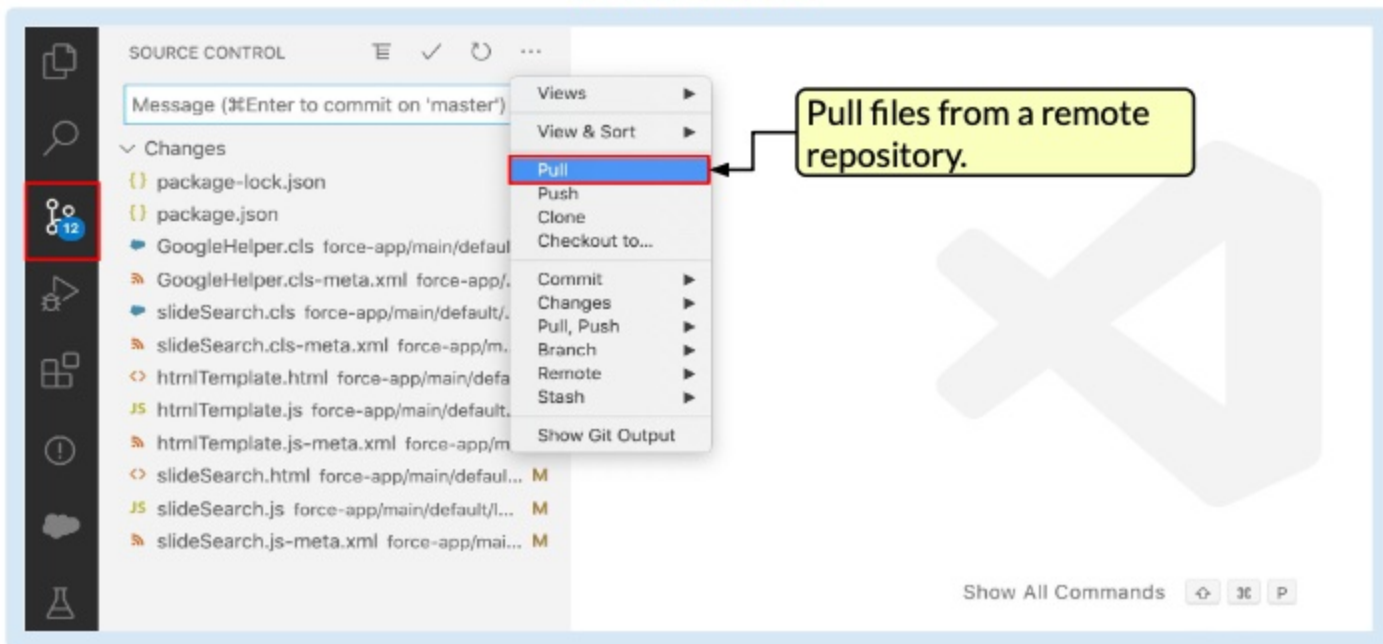
Authorizing an Org

The command to **log in** or **authorize** a connection with an org can be selected from the **command palette** in Visual Studio Code.



Pull Files from a Repository

Salesforce **metadata** and **source code** can be pulled from a remote **code repository** into the **local machine**.



Deploy or Retrieve Metadata

The following **describes** when to use the **commands**, `force:source:deploy` and `force:mdapi:deploy`:



DEVELOPMENT

'`force:source:deploy`' is intended for working with a sandbox org for **development** or **local testing**.



DEPLOYMENT

'`force:mdapi:deploy`' is intended for deploying code for **integration testing**, **staging**, or **production**.

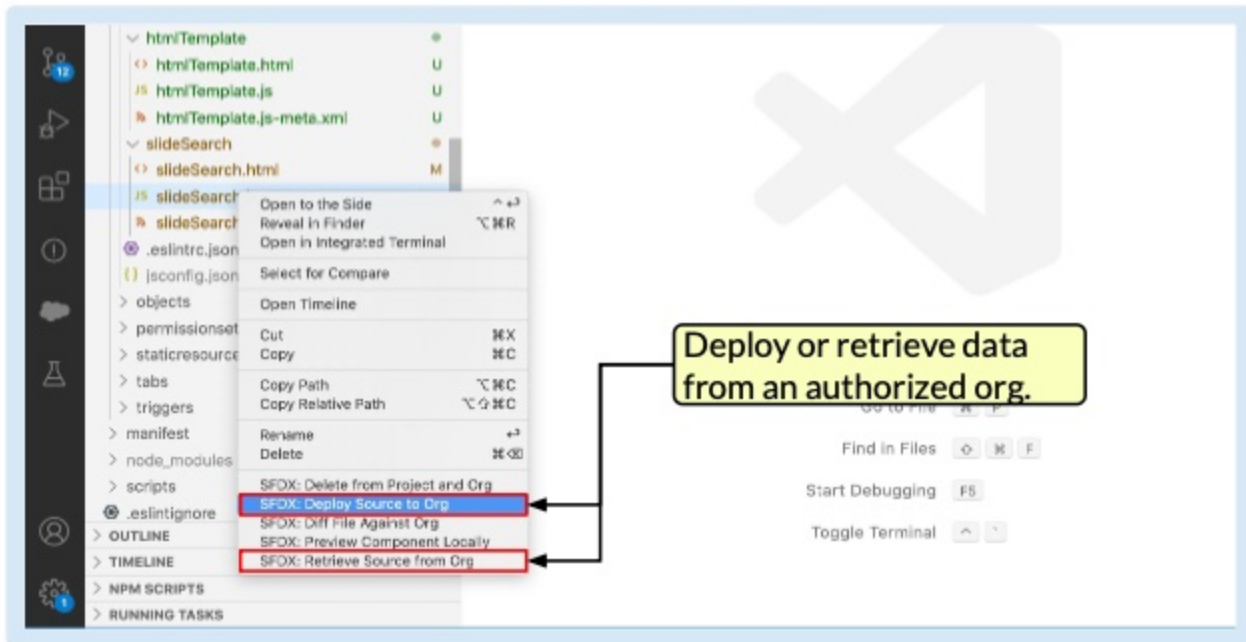


CONVERT SOURCE

'`force:mdapi:deploy`' requires the **source code** to be converted to **metadata format** using '`sfdx force:source:convert`'.

Deploy or Retrieve Metadata

Right-clicking on a file in a Salesforce DX project also **displays options** for **retrieving** or **deploying** the file to a connected org.



Code Builder

Table of Contents



Code Builder

Code Builder is an integrated development environment that runs entirely on the web browser and inherits all the capabilities of **Visual Studio Code**, **Salesforce Extensions for VS Code**, and **Salesforce CLI**.

The screenshot shows the 'SETUP Code Builder' interface. On the left, under 'Get started with Code Builder', there are two sections: 'Enable Code Builder' with a toggle switch labeled 'enabled', and 'Code Builder is ready to install' with an 'Install Package' button. On the right, under 'Enable Code Builder to:', there is a yellow callout box with text. Two arrows point from the callout box to the 'enabled' toggle and the 'Install Package' button.

SETUP Code Builder

Get started with Code Builder

Enable Code Builder

Code Builder is ready to install

After you install the package, be sure to complete the [configuration steps](#) in the Code Builder documentation.

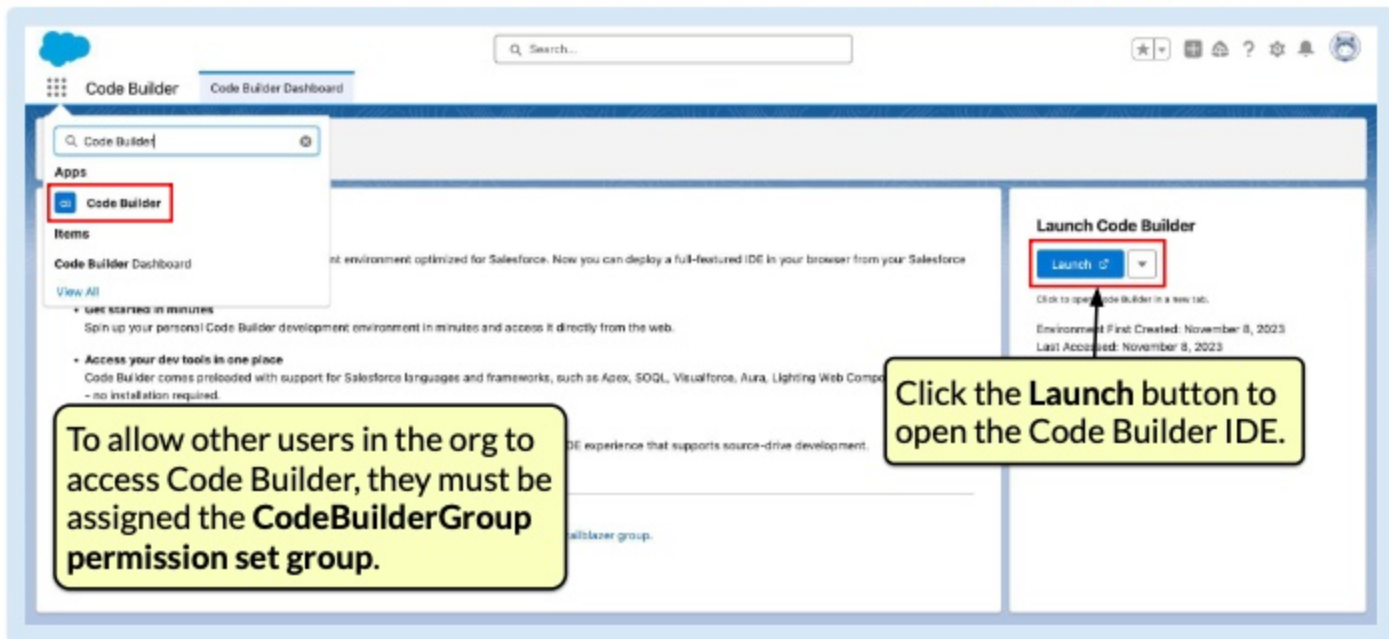
Install Package

Enable Code Builder to:

To use Code Builder, the option is enabled in the production org. Then, the Code Builder managed package is installed.

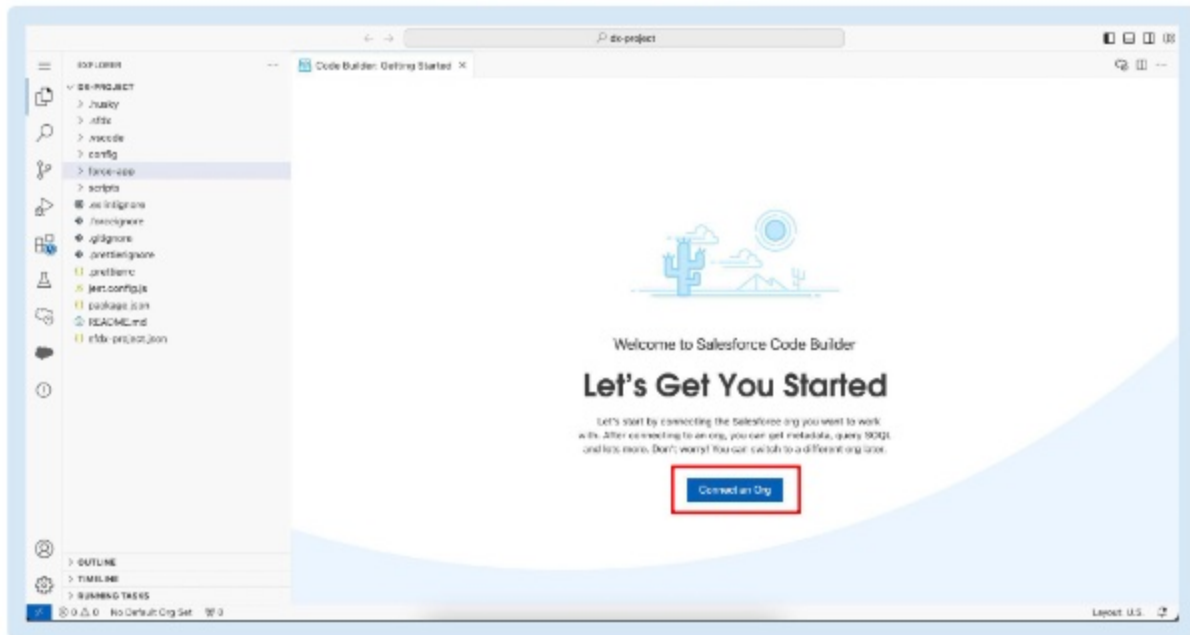
Launch Code Builder

After the Code Builder package is installed in the org, the **Code Builder app** for launching the integrated development environment can be accessed from the **App Launcher** menu.



Connect Code Builder

After Code Builder is first launched, a **Connect an Org** button is provided for authorizing access to any Salesforce org.



Code Builder

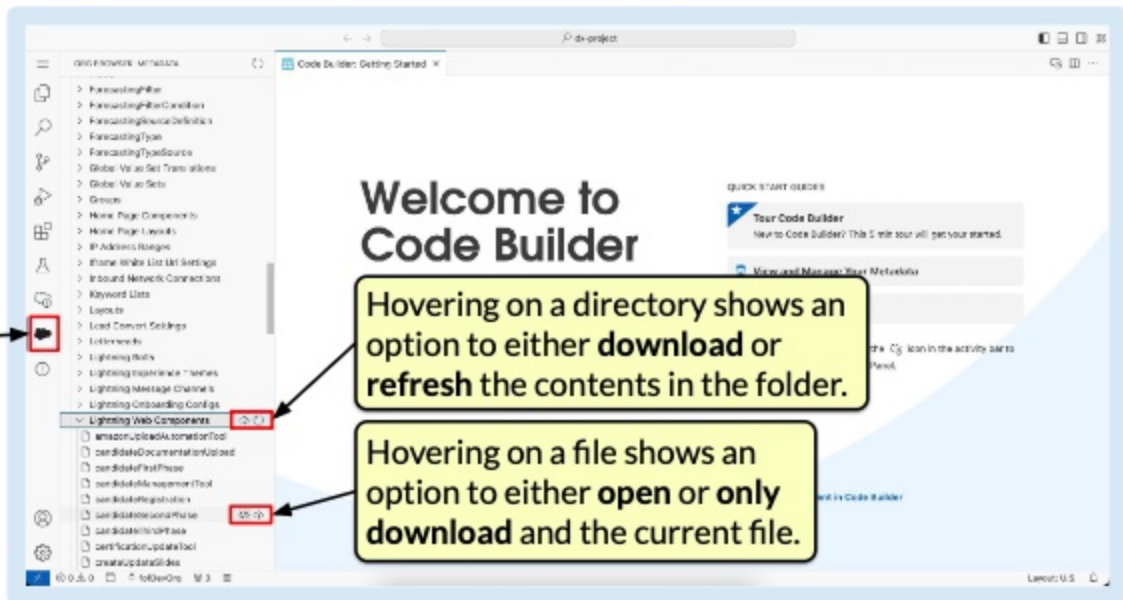
The **Org Browser** panel in Code Builder can be used to **browse** and **download** metadata into the Code Builder project from the connected org without using a **manifest file**.

The **Org Browser** panel can be accessed from the Activity Bar.

Welcome to Code Builder

Hovering on a directory shows an option to either **download** or **refresh** the contents in the folder.

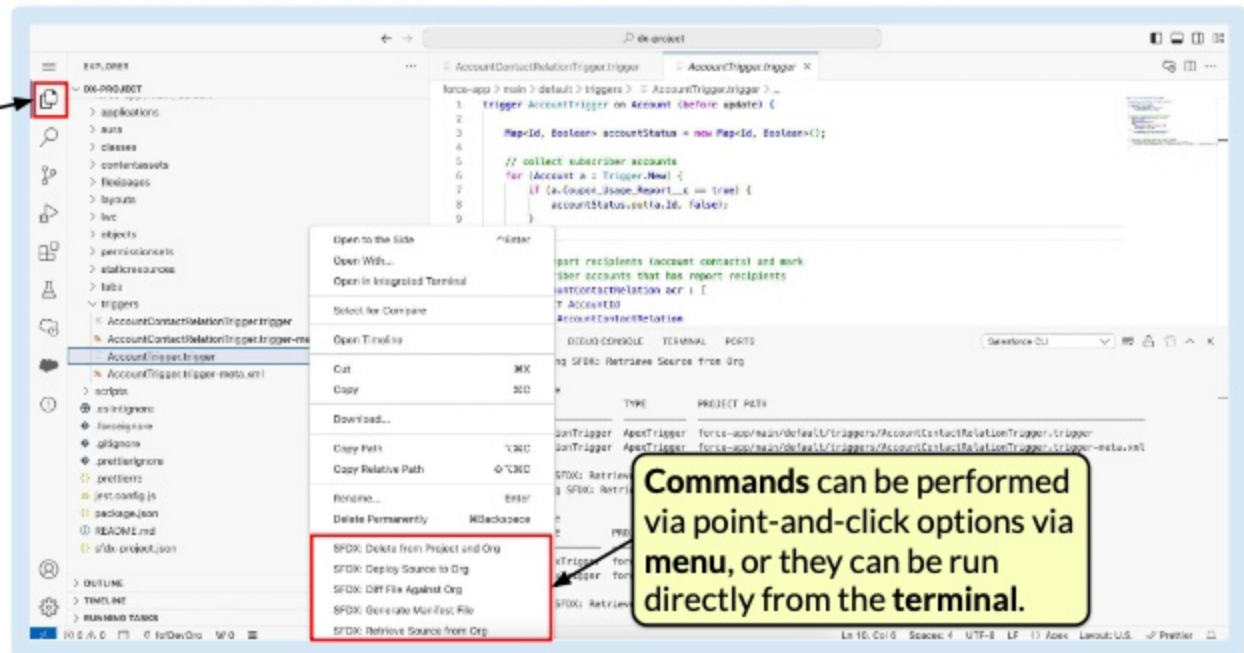
Hovering on a file shows an option to either **open** or **only download** and the current file.



Code Builder

The **Explorer panel** in Code Builder provides options for working with metadata such as **creating**, **updating**, **deleting**, or **deploying** source files to the connected org.

The **Explorer panel** can be accessed from the **Activity Bar**.



Code Builder Extensions

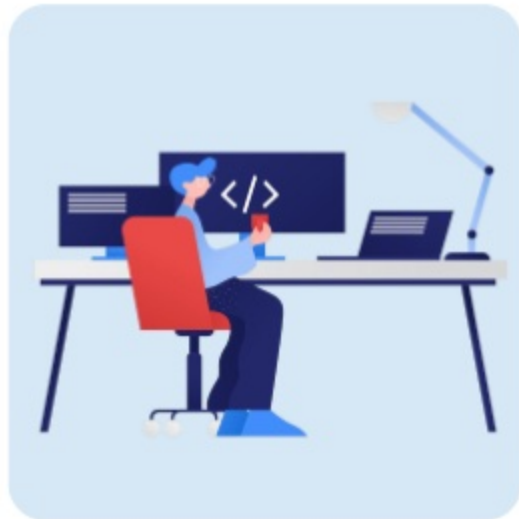
The following are **some considerations** regarding extensions in Code Builder.

◆ PACKAGE

Code Builder is installed as a **managed package** in the current org and already comes with everything needed for Salesforce development. **No installation** is required in the local machine. In addition, Salesforce maintains and automatically performs the necessary updates for **Salesforce extensions** that are pre-installed in Code Builder.

◆ MARKETPLACE

The Microsoft Marketplace cannot be accessed from Code Builder. Instead, extensions can be installed from **[Open VSX](#)**. So, the available extensions that can be installed may differ.



Local Dev

Table of Contents



Enable Real-Time Preview

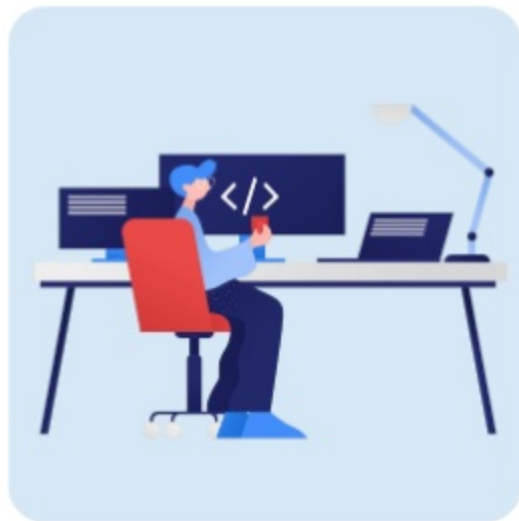
When developing **Lightning web components**, Salesforce allows **real-time preview** of the changes made to the component without the need to deploy code to the org, significantly improving development time.

❖ LOCAL DEV

Local Dev is a feature that allows developers to preview changes made to a Lightning web component in real-time. To enable this feature, the following are required:

- **Salesforce CLI** must be installed in the local machine.
- The **Local Dev plugin** ([@salesforce/plugin-lightning-dev](#)) must be installed in the local machine.
- The **Local Dev** settings must be enabled in the target org (**Setup > Local Dev > Enable Local Dev**).

Note: If real-time preview is not working, there may be a need to enable Lightning web security and clear the browser cache (for desktop previews).



Launch the Preview Mode

To launch the preview mode, the “**sf lightning dev app**” command is ran from the command line which will then ask to specify which **device** to view from and the **Lightning Experience app** to load.

The available devices for previewing are **desktop** and **mobile** (Android or iOS).

After choosing the device, the **Lightning app** to open is then selected. So, to preview the **Lightning web component** in this case, it **must be added** or, at least, **viewed from** this Lightning app.

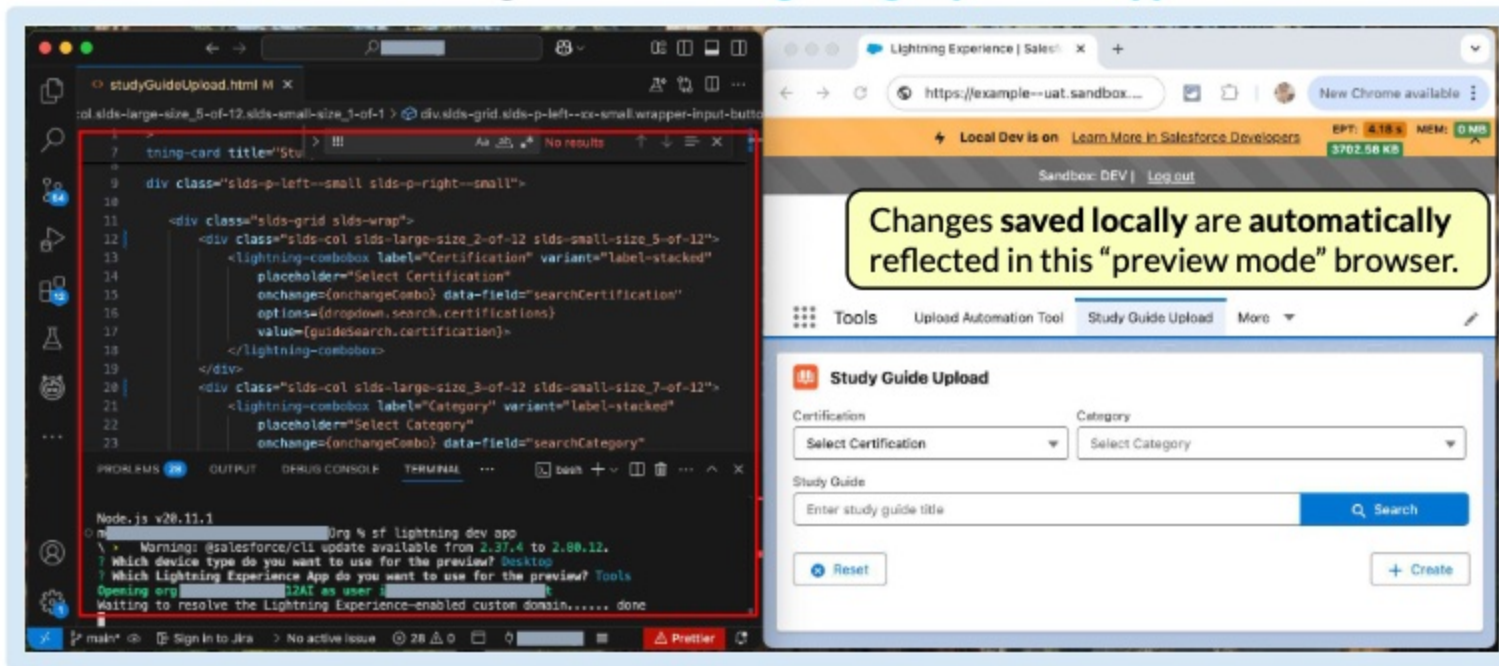
In this example, the Lightning web component is added to a Lightning app called “Tools”. Choosing the Lightning app **launches the browser automatically** and **opens the app** in preview mode.

```
o m [redacted]Org % sf lightning dev app
> Warning: @salesforce/cli update available from 2.37.4 to 2.80.12.
? Which device type do you want to use for the preview? (Use arrow keys)
> Desktop
  Mobile - Android
  Mobile - iOS
```

```
o m [redacted]Org % sf lightning dev app
> Warning: @salesforce/cli update available from 2.37.4 to 2.80.12.
? Which device type do you want to use for the preview? Desktop
? Which Lightning Experience App do you want to use for the preview?
  My Service Journey
  Policy Center
  CUS
> Tools
  Sales Leadership
  Sales Operations
  Feedback
```

Enable Real-Time Preview

The **Lightning web component** that is being previewed in this example is called "Study Guide Upload", which is added as a navigation tab to a **Lightning Experience App** called "Tools".

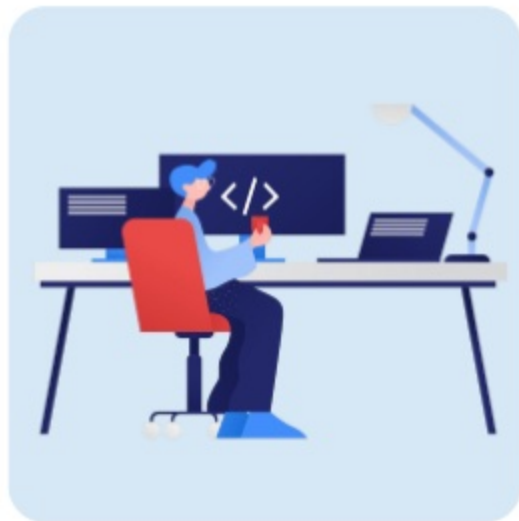


Real-Time Preview Limitations

Real-time preview is **not supported for all types of changes** made to a Lightning web component. For example, only the following types of changes are automatically reflected in the previewing device.

◆ SUPPORTED CHANGES

- Basic **HTML revisions**
 - Such as changing attributes, adjusting layout, etc.
- **New reference** to a component
 - Such as adding a `<lightning-card>` to the markup.
- Basic **CSS revisions**
 - Such as changing font-size, colors, etc.
- Importing a new **CSS-only component**
 - Such as adding `@import "c/myAdditionalStyles";`
- JavaScript changes in component methods that **don't affect the component's public API**



Scratch Orgs

[Table of Contents](#)



What are Scratch Orgs

Scratch orgs are temporary **configurable Salesforce environments** that can be **quickly generated using Salesforce CLI** to be used as a personal development environment, or for the following:

USE CASES

- Starting a **new project**
- Starting a **new feature branch**
- Starting **automated testing**
- Testing a **new feature**
- Performing **development tasks** directly in an org
- **Starting from "scratch"** with a fresh new org

Scratch Org Configuration

Salesforce DX enables developers to create an org with a **specific edition and set of features that can be replicated** by using scratch orgs.



EDITION / FEATURES

Scratch orgs can be customized with various **Salesforce editions** and tailored to include only the **necessary features** and **preferences**.



DEFINITION FILE

Scratch org configuration can be shared to allow developers replicate each other's scratch orgs to ensure that the team is working on the "same" environment.



ORG SNAPSHOTS

A **snapshot** of a scratch org's configuration can be captured. A snapshot represents a **point-in-time copy** of a scratch org that can be used to **replicate** the org.

Scratch Orgs vs Sandbox Orgs

While **source development** use cases should be performed in scratch orgs, **release and deployment testing** should be performed in sandbox orgs. Here are the other main differences:



LIMITED

Scratch orgs have **limited storage size** and (short) **life span**



PURPOSE

Scratch orgs are **not designed to replace** sandbox orgs

Scratch Orgs vs Sandbox Orgs



DEVELOPMENT

Scratch orgs are intended for **developing customizations** in modules or packages



TESTING

Scratch orgs are designed for **testing an individual module or package**



USE CASES

Sandbox orgs are used for **user acceptance testing**, as a **staging environment**, and for **continuous delivery testing**.

Salesforce CLI

Table of Contents



Salesforce CLI

Salesforce CLI is a **command-line interface** built on top of the metadata API that is used for performing tasks all throughout the different phases of an **application development lifecycle**.



Authorizing
sandboxes



Creating and
managing **DX projects**



Importing and
exporting **test data**



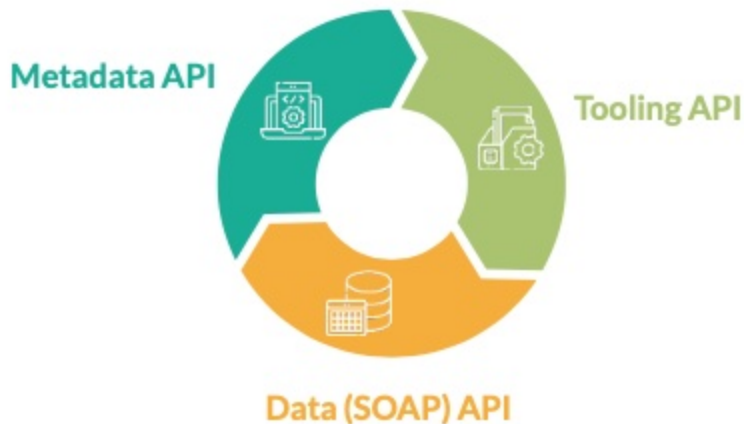
Retrieving and
deploying **metadata**



Running and
automating **tests**

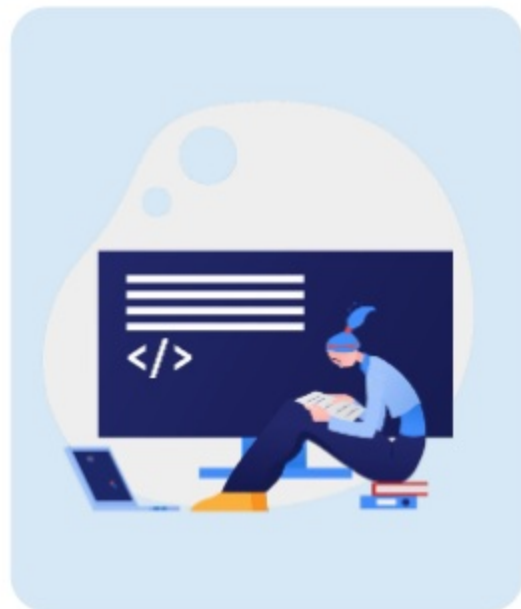
Salesforce CLI Capabilities

Salesforce CLI combines capabilities **across several Salesforce APIs and tools** and enables development tasks from all the important APIs available in one place.



Salesforce CLI

Salesforce CLI is the **command-line interface** for Salesforce **APIs** and carries out **instructions** for a Salesforce org.



❖ METADATA

Metadata can be **exported and imported**.

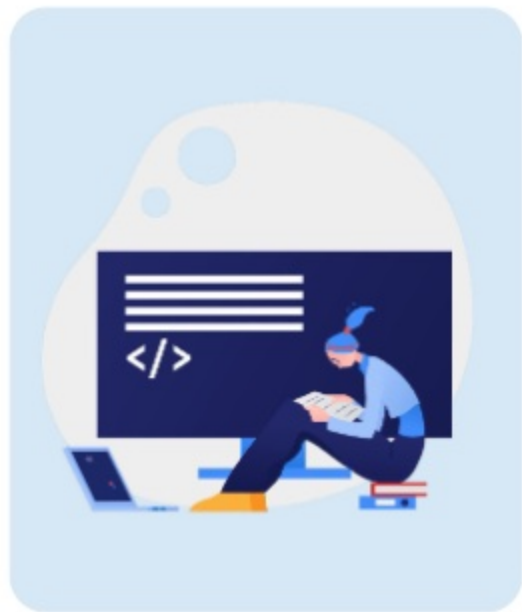
❖ COMMANDS & TASKS

It allows deployments using **scripted commands** and **automated tasks**.

❖ INTERACTIVE LOGIN

When **connecting** an org, Salesforce CLI automatically opens the standard Salesforce login page for the user to **log in** and **authorize** the connection.

Salesforce CLI



◆ NOT STORED

No username/password is stored in the file system.

◆ FIREWALL

Logging in behind a firewall can be difficult.

Salesforce CLI Commands

The following lists some operations where **Salesforce CLI can be used** to perform them.



Authorize a default Dev Hub org and assign an alias:

```
sf org login web --set-default-dev-hub --alias my-dev-hub
```



Create a scratch org using a definition file, assign an alias, and set the default username:

```
sf org create scratch --definition-file config/project-scratch-def.json --alias MyScratchOrg
```



Generate a Lightning web component:

```
sf lightning generate component --type lwc --name myApp --output-dir force-app/main/default/lwc
```



Run Apex tests:

```
sf apex run test --result-format human
```



Open the scratch org and view the pushed metadata:

```
sf org open
```

Salesforce CLI Commands



Generate a project with a default manifest (package.xml) file
sf project generate --name myProject --manifest



Retrieve metadata in source format from an org
sf project retrieve start --source-dir path/to/source --target-org myOrg



Deploy files in the *force-app* directory to a specified org:
sf project deploy start --source-dir path/to/source --target-org myOrg



Preview a deployment to determine any conflicts or issues
sf project deploy preview --source-dir force-app



Preview a retrieve to determine any conflicts or issues
sf project retrieve preview

Salesforce CLI

Salesforce CLI commands start with **sf** (or **sfdx**) such as the *sf help* command in this example.

```
Last login: Wed Apr  3 15:39:53 on ttys004
```

```
~ % sf help
```

```
The Salesforce CLI
```

VERSION

```
@salesforce/cli/2.34.7 darwin-arm64 node-v20.11.1
```

USAGE

```
$ sf [COMMAND]
```

TOPICS

alias	Use the alias commands to manage your aliases.
analytics	Work with analytics assets.
apex	Use the apex commands to create Apex classes, execute anonymous blocks, view your logs, run Apex tests, and view Apex test results.
cmdt	Generate a field for a custom metadata type based on the provided field type.
community	Create an Experience Cloud site using a template.
config	Commands to configure Salesforce CLI.
data	Manage records in your org.
deploy	Deploy a Salesforce Function to an org from your local project.
dev	Audit messages in a plugin's messages directory to locate unused messages and missing messages that have references in source code.
env	Add a Heroku user as a collaborator on this Functions account, allowing them to attach Heroku add-ons to compute environments.

Salesforce Extensions for Visual Studio Code and Code Builder use Salesforce CLI to interact with the org.

Example Development Environment

The following illustrates a development environment example **using Salesforce DX**.



Scratch orgs, which require a *Dev Hub org*, can be created using Salesforce CLI for development and testing. Also, **metadata can be deployed or retrieved** between the local machine and a sandbox org used for staging or user acceptance testing.

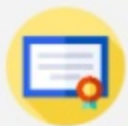
Partners, Packaging, and AppExchange

Table of Contents



AppExchange Partner Tools

For **AppExchange** partners, or ISVs (Independent Software Vendors), Salesforce provides tools for **packaging, managing, and deploying** solutions in the AppExchange platform.



ENVIRONMENT HUB

Allows partners to **view, connect, create, and log in** to multiple Salesforce organizations from a single location.



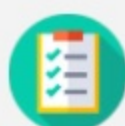
LICENSE MANAGEMENT APP (LMA)

Used to manage **leads** and **licenses** for custom solutions distributed and sold on the AppExchange platform.



CHECKOUT MANAGEMENT APP (CMA)

Provides access to a dashboard that displays AppExchange Checkout data such as **revenues** and **subscriptions**.



CHANNEL ORDER APP (COA)

Enables partners to **create, submit, and track orders** of customers such as a product purchase, adding of users, or renewal of contracts.

App Packaging Considerations

Here are some **considerations** when **distributing** an app, Lightning component, or other solutions.

◆ PACKAGE MANAGER

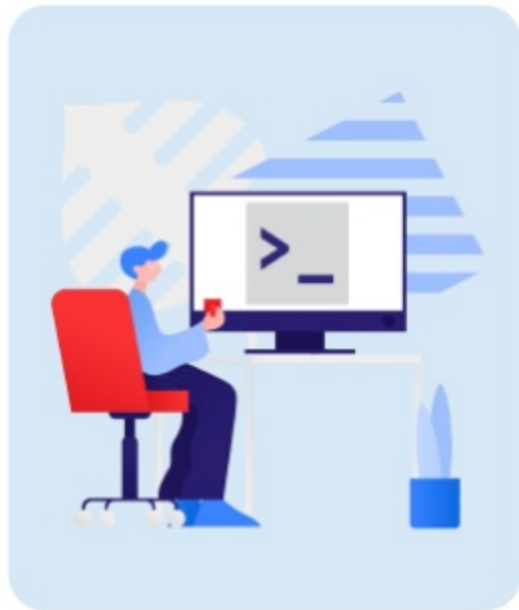
The Package Manager tool in Setup can be used to create **managed packages** as well as **unmanaged packages**.

◆ SECURITY REVIEW

Each product will need to be reviewed by a **Product Security team** in Salesforce and pass before it can be published on the AppExchange.

◆ LIGHTNING READY

New apps, whether paid or free, to be published in AppExchange should be Lightning Ready in order to go through the **security review**.



DevOps Center

[Table of Contents](#)



DevOps Center

The DevOps Center is a declarative tool that can be used to not only **manage and deploy changes** between orgs but also **track changes**. DevOps Center can be enabled in Setup.

Get started with DevOps Center

Enable DevOps Center



DevOps Center is ready to install

[Install Package...](#)

After you install the package, be sure to complete the [configuration steps](#).

After the DevOps Center is enabled, the **DevOpsCenter** package should be installed.

Installed Packages

[Help for this Page](#)

On AppExchange you can browse, test drive, download, and install pre-built apps and components right into your salesforce.com environment. [Learn More about Installing Packages](#).

Apps and components are installed in packages. Any custom apps, tabs, and custom objects are initially marked as "In Development" and are not deployed to your users. This allows you to test and customize before deploying. You can deploy the components individually using the other features in setup or as a group by clicking Deploy.

Depending on the links next to an installed package, you can take different actions from this page.

To remove a package, click **Uninstall**. To manage your package licenses, click **Manage Licenses**.



Installed Packages

Action	Package Name	Publisher	Version Number	Namespace Prefix	Status	Allowed Licenses	Used Licenses	Enabled for Platform Integrations	Expires
Uninstall Manage Licenses	Slack	Slack	1.08	slack2	Active	Unlimited	0		Does Not Expire
Description Consolidated AppExchange Dashboards for 1 Installation: Sales, Marketing, Support, Adoption and more									
Uninstall	DevOps Center	Salesforce	5.5	sf_devops	Free	N/A	N/A	N/A	N/A

DevOps Center

The DevOps Center requires creating a **Connected App** in order to make the **DevOps Center app** available in the **App Launcher**. Access to the DevOps Center app requires additional permissions.

The image shows two overlapping screenshots from the Salesforce interface. The background screenshot is the 'Connected App Detail' page for 'DevOps Center'. It features a blue circular icon with a white gear and arrows. The 'System Info' section shows it was installed by Martin Gessner on 4/14/2023 at 5:29 PM. The 'Basic Information' section shows the start URL as /sf_devops/DevOpsCenter.app. The foreground screenshot is the 'App Launcher' with 'DevOps' in the search bar and 'DevOps Center' highlighted in the results list.

Connected App Detail

DevOps Center

Connected App Detail

Version 1

Description Manage your development and release processes

System Info

Installed By	Martin Gessner	Installed Date	4/14/2023 5:29 PM
Last Modified By	Martin Gessner	Last Modified Date	4/14/2023 5:29 PM

Basic Information

Info URL		Start URL	/sf_devops/DevOpsCenter.app
		Mobile Start URL	

App Launcher

Search: DevOps

Apps

- DevOps Center

Items

No results

[View All](#)

DevOps Center

DevOps Center users are required to have a **GitHub account** and access to the **GitHub repository** where Salesforce metadata are **stored, managed, and tracked**.

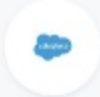
 Salesforce Platform

 **DevOps Center**

All Projects
Displaying 1 of 1

Name	Owner	Repository URL	Release Environment	All Environments
Focus on Force Production Org	Martin Gessner	https://github.com/focusonforce/fof_prd_org	https://fof.my.salesforce.com	0 Dev / 1 Other

When creating a new project in the DevOps Center, **authorization** to access the GitHub repository is required.



Authorize Salesforce Integration Application

 **Salesforce Integration Application** by [Salesforce Platform](#)
wants to access your focusonforce account

 **Repositories**
Public and private

 **Personal user data**
Full access

Cancel

Authorize forcedotcom

Authorizing will redirect to
<https://login.salesforce.com>

DevOps Center

The **DevOps Center Pipeline** is used to manage the environments used in the release process of a project. Development orgs are required to have **source tracking enabled**.

The screenshot displays the Salesforce DevOps Center interface. On the left, a sidebar contains navigation links: 'View all projects', 'Project: Focus on Force Production Org', 'Work Items', 'Pipeline' (highlighted), and 'Settings'. The main area is titled 'Pipeline' and has tabs for 'Environments' and 'Activity History'. A yellow callout box in the top right corner states: 'Similar to change sets validation, a **validate-only deployment** can be performed to determine any potential issues that will be encountered in the actual deployment.' Below this, a 'Not Ready' status with an 'Activate' button is visible. The 'Environments' tab shows a sequence of development environments connected by arrows. A red box highlights the configuration for two stages. The first stage, 'LIAT', has an 'Add Development Environment' button. The second stage, 'Production', is configured with 'Focus on Force Production Org Release' as the environment and 'Existing branch main' as the branch. A yellow callout box at the bottom of the red box states: 'Environments can be easily **connected, disconnected, removed, and even swapped.**'

Similar to change sets validation, a **validate-only deployment** can be performed to determine any potential issues that will be encountered in the actual deployment.

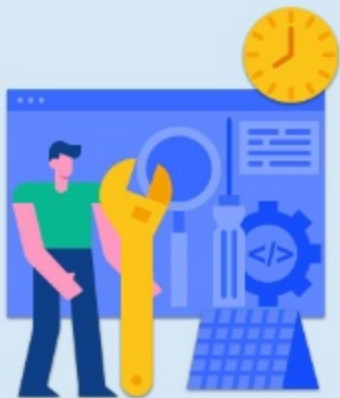
Environments can be easily **connected, disconnected, removed, and even swapped.**

Scenarios and Solutions

Table of Contents



Scenarios and Deployment Solutions



SCENARIO

Developers would like to automatically create and use temporary orgs as personal development environments based on specific configurations.



SOLUTION

Scratch orgs are short-lived orgs that are generated automatically using **Salesforce CLI** and are used for developing and testing applications. A configuration file can be shared to build scratch orgs with pre-configured metadata.

Scenarios and Deployment Solutions



SCENARIO

Developers have been using spreadsheets to track changes made to Lightning components as well Visualforce pages in their org and need to find an appropriate and efficient change tracking solution.



SOLUTION

A **version control system** should be used to track and manage changes made to org metadata. Typically, Salesforce CLI is used to retrieve org metadata into a Salesforce DX project. Then a code repository that contains the project is used for storing files and tracking changes.



Scenarios and Deployment Solutions



SCENARIO

A developer is assigned a separate development sandbox for building a new feature. The developer needs to retrieve source code from the sandbox org as well as deploy to it.



SOLUTION

Salesforce CLI can be used to interact with **sandbox orgs** for **development** and **local testing**. By using the Salesforce Extensions for VS Code, or Salesforce CLI interface directly, metadata can be conveniently **deployed** or **retrieved** between the org and local machine.

Scenarios and Deployment Solutions



SCENARIO

A developer has worked on an update and tested it in a development org to fix a bug that was encountered in production. It is ready to go through integration testing and staging, and then deployment to the production org.



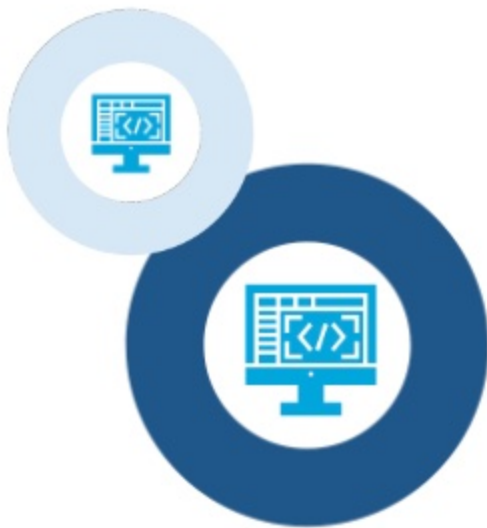
SOLUTION

For deploying changes to **integration**, **staging**, and **production** orgs, it is recommended to perform the deployment using the **'force:mdapi:deploy'** command. Unlike **'force:source:deploy'**, this command rolls back the entire deployment when any component has an error, and helps **maintain** the **integrity** of the target environments.



Learn More

-  [Change Set Development Model](#)
-  [Package Development Model](#)
-  [Org Development Model](#)
-  [How Salesforce DX Changes the Way You Work](#)
-  [Increase Productivity with Integrated Tools](#)
-  [Salesforce Extensions for Visual Studio Code](#)



Learn More

-  [Salesforce DX and Source Driven Development](#)
-  [Getting Started With Scratch Orgs](#)
-  [Keep Your Org in Tip-Top Shape with Deployment Best Practices](#)
-  [Force.com Migration Tool](#)
-  [Force.com CLI](#)

